

4

Working with nonrelational data

This chapter covers

- Persisting data to Cassandra
- Data modeling in Cassandra
- Working with document data in MongoDB

They say that variety is the spice of life.

You probably have a favorite flavor of ice cream. It's that one flavor that you choose the most often because it satisfies that creamy craving more than any other. But most people, despite having a favorite flavor, try different flavors from time to time to mix things up.

Databases are like ice cream. For decades, the relational database has been the favorite flavor for storing data. But these days, we have more options available than ever before. So-called “NoSQL” databases (<https://aws.amazon.com/nosql/>) offer different concepts and structures in which data can be stored. And although the choice may still be somewhat based on taste, some databases are better suited for persisting different kinds of data than others.

Fortunately, Spring Data has you covered for many of the NoSQL databases, including MongoDB, Cassandra, Couchbase, Neo4j, Redis, and many more. And fortunately, the programming model is nearly identical, regardless of which database you choose.

There's not enough space in this chapter to cover all of the databases that Spring Data supports. But to give you a sample of Spring Data's other "flavors," we'll look at two popular NoSQL databases, Cassandra and MongoDB, and see how to create repositories to persist data to them. Let's start by looking at how to create Cassandra repositories with Spring Data.

4.1 Working with Cassandra repositories

Cassandra is a distributed, high-performance, always available, eventually consistent, partitioned-column-store, NoSQL database.

That's a mouthful of adjectives to describe a database, but each one accurately speaks to the power of working with Cassandra. To put it in simpler terms, Cassandra deals in rows of data written to tables, which are partitioned across one-to-many distributed nodes. No single node carries all the data, but any given row may be replicated across multiple nodes, thus eliminating any single point of failure.

Spring Data Cassandra provides automatic repository support for the Cassandra database that's quite similar to—and yet quite different from—what's offered by Spring Data JPA for relational databases. In addition, Spring Data Cassandra offers annotations for mapping application domain types to the backing database structures.

Before we explore Cassandra any further, it's important to understand that although Cassandra shares many concepts similar to relational databases like Oracle and SQL Server, Cassandra isn't a relational database and is in many ways quite a different beast. I'll explain the idiosyncrasies of Cassandra as they pertain to working with Spring Data. But I encourage you to read Cassandra's own documentation (<http://cassandra.apache.org/doc/latest/>) for a thorough understanding of what makes it tick.

Let's get started by enabling Spring Data Cassandra in the Taco Cloud project.

4.1.1 Enabling Spring Data Cassandra

To get started using Spring Data Cassandra, you'll need to add the Spring Boot starter dependency for nonreactive Spring Data Cassandra. There are actually two separate Spring Data Cassandra starter dependencies to choose from: one for reactive data persistence and one for standard, nonreactive persistence.

We'll talk more about writing reactive repositories later in chapter 15. For now, though, we'll use the nonreactive starter in our build as shown here:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-cassandra</artifactId>
</dependency>
```

This dependency is also available from the Initializr by checking the Cassandra check box.

It's important to understand that this dependency is in lieu of the Spring Data JPA starter or Spring Data JDBC dependencies we used in the previous chapter. Instead of

persisting Taco Cloud data to a relational database with JPA or JDBC, you'll be using Spring Data to persist data to a Cassandra database. Therefore, you'll want to remove the Spring Data JPA or Spring Data JDBC starter dependencies and any relational database dependencies (such as JDBC drivers or the H2 dependency) from the build.

The Spring Data Cassandra starter dependency brings a handful of dependencies to the project, specifically, the Spring Data Cassandra library. As a result of Spring Data Cassandra being in the runtime classpath, autoconfiguration for creating Cassandra repositories is triggered. This means you're able to begin writing Cassandra repositories with minimal explicit configuration.

Cassandra operates as a cluster of nodes that together act as a complete database system. If you don't already have a Cassandra cluster to work with, you can start a single-node cluster for development purposes using Docker like this:

```
$ docker network create cassandra-net
$ docker run --name my-cassandra \
    --network cassandra-net \
    -p 9042:9042 \
    -d cassandra:latest
```

This starts the single-node cluster and exposes the node's port (9042) on the host machine so that your application can access it.

You'll need to provide a small amount of configuration, though. At the very least, you'll need to configure the name of a keyspace within which your repositories will operate. To do that, you'll first need to create such a keyspace.

NOTE In Cassandra, a *keyspace* is a grouping of tables in a Cassandra node. It's roughly analogous to how tables, views, and constraints are grouped in a relational database.

Although it's possible to configure Spring Data Cassandra to create the keyspace automatically, it's typically much easier to manually create it yourself (or to use an existing keyspace). Using the Cassandra CQL (Cassandra Query Language) shell, you can create a keyspace for the Taco Cloud application. You can start the CQL shell using Docker like this:

```
$ docker run -it --network cassandra-net --rm cassandra cqlsh my-cassandra
```

NOTE If this command fails to start up the CQL shell with an error indicating "Unable to connect to any servers," wait a minute or two and try again. You need to be sure that the Cassandra cluster is fully started before the CQL shell can connect to it.

When the shell is ready, use the create keyspace command like this:

```
cqlsh> create keyspace tacocloud
... with replication={'class': 'SimpleStrategy', 'replication_factor': 1}
... and durable_writes=true;
```

Put simply, this will create a keyspace named `tacocloud` with simple replication and durable writes. By setting the replication factor to 1, you ask Cassandra to keep one copy of each row. The replication strategy determines how replication is handled. The `SimpleStrategy` replication strategy is fine for single data center use (and for demo code), but you might consider the `NetworkTopologyStrategy` if you have your Cassandra cluster spread across multiple data centers. I refer you to the Cassandra documentation for more details of how replication strategies work and alternative ways of creating keyspaces.

Now that you've created a keyspace, you need to configure the `spring.data.cassandra.keyspace-name` property to tell Spring Data Cassandra to use that keyspace, as shown next:

```
spring:
  data:
    cassandra:
      keyspace-name: taco_cloud
      schema-action: recreate
      local-datacenter: datacenter1
```

Here, you also set the `spring.data.cassandra.schema-action` to `recreate`. This setting is very useful for development purposes because it ensures that any tables and user-defined types will be dropped and recreated every time the application starts. The default value, `none`, takes no action against the schema and is useful in production settings where you'd rather not drop all tables whenever an application starts up.

Finally, the `spring.data.cassandra.local-datacenter` property identifies the name of the local data center for purposes of setting Cassandra's load-balancing policy. In a single-node setup, `"datacenter1"` is the value to use. For more information on Cassandra load-balancing policies and how to set the local data center, see the DataStax Cassandra driver's reference documentation (<http://mng.bz/XrQM>).

These are the only properties you'll need for working with a locally running Cassandra database. In addition to these two properties, however, you may wish to set others, depending on how you've configured your Cassandra cluster.

By default, Spring Data Cassandra assumes that Cassandra is running locally and listening on port 9042. If that's not the case, as in a production setting, you may want to set the `spring.data.cassandra.contact-points` and `spring.data.cassandra.port` properties as follows:

```
spring:
  data:
    cassandra:
      keyspace-name: tacocloud
      local-datacenter: datacenter1
      contact-points:
        - casshost-1.tacocloud.com
        - casshost-2.tacocloud.com
        - casshost-3.tacocloud.com
      port: 9043
```

Notice that the `spring.data.cassandra.contact-points` property is where you identify the hostname(s) of Cassandra. A contact point is the host where a Cassandra node is running. By default, it's set to `localhost`, but you can set it to a list of hostnames. It will try each contact point until it's able to connect to one. This is to ensure that there's no single point of failure in the Cassandra cluster and that the application will be able to connect with the cluster through one of the given contact points.

You may also need to specify a username and password for your Cassandra cluster. This can be done by setting the `spring.data.cassandra.username` and `spring.data.cassandra.password` properties, as shown next:

```
spring:
  data:
    cassandra:
      ...
      username: tacocloud
      password: s3cr3tP455w0rd
```

These are the only properties you'll need for working with a locally running Cassandra database. In addition to these two properties, however, you may wish to set others, depending on how you've configured your Cassandra cluster.

Now that Spring Data Cassandra is enabled and configured in your project, you're almost ready to map your domain types to Cassandra tables and write repositories. But first, let's step back and consider a few basic points of Cassandra data modeling.

4.1.2 Understanding Cassandra data modeling

As I mentioned, Cassandra is quite different from a relational database. Before you can start mapping your domain types to Cassandra tables, it's important to understand a few of the ways that Cassandra data modeling is different from how you might model your data for persistence in a relational database.

A few of the most important things to understand about Cassandra data modeling follow:

- Cassandra tables may have any number of columns, but not all rows will necessarily use all of those columns.
- Cassandra databases are split across multiple partitions. Any row in a given table may be managed by one or more partitions, but it's unlikely that all partitions will have all rows.
- A Cassandra table has two kinds of keys: partition keys and clustering keys. Hash operations are performed on each row's partition key to determine which partition(s) that row will be managed by. Clustering keys determine the order in which the rows are maintained within a partition (not necessarily the order in which they may appear in the results of a query). Refer to Cassandra documentation (<http://mng.bz/yJ6E>) for a more detailed explanation of data modeling in Cassandra, including partitions, clusters, and their respective keys.

- Cassandra is highly optimized for read operations. As such, it's common and desirable for tables to be highly denormalized and for data to be duplicated across multiple tables. (For example, customer information may be kept in a customer table as well as duplicated in a table containing orders placed by customers.)

Suffice it to say that adapting the Taco Cloud domain types to work with Cassandra won't be a matter of simply swapping out a few JPA annotations for Cassandra annotations. You'll have to rethink how you model the data.

4.1.3 Mapping domain types for Cassandra persistence

In chapter 3, you marked up your domain types (Taco, Ingredient, TacoOrder, and so on) with annotations provided by the JPA specification. These annotations mapped your domain types as entities to be persisted to a relational database. Although those annotations won't work for Cassandra persistence, Spring Data Cassandra provides its own set of mapping annotations for a similar purpose.

Let's start with the Ingredient class, because it's the simplest to map for Cassandra. The new Cassandra-ready Ingredient class looks like this:

```
package tacos;

import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Table("ingredients")
public class Ingredient {

    @PrimaryKey
    private String id;
    private String name;
    private Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

The Ingredient class seems to contradict everything I said about just swapping out a few annotations. Rather than annotating the class with `@Entity` as you did for JPA

persistence, it's annotated with `@Table` to indicate that ingredients should be persisted to a table named `ingredients`. And rather than annotate the `id` property with `@Id`, this time it's annotated with `@PrimaryKey`. So far, it seems that you're only swapping out a few annotations.

But don't let the `Ingredient` mapping fool you. The `Ingredient` class is one of your simplest domain types. Things get more interesting when you map the `Taco` class for Cassandra persistence, as shown in the next listing.

Listing 4.1 Annotating the `Taco` class for Cassandra persistence

```
package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import org.springframework.data.cassandra.core.cql.Ordering;
import org.springframework.data.cassandra.core.cql.PrimaryKeyType;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKeyColumn;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;

import lombok.Data;

@Data
@Table("tacos")
public class Taco {

    @PrimaryKeyColumn(type=PrimaryKeyType.PARTITIONED)
    private UUID id = Uuids.timeBased();

    @NotNull
    @Size(min = 5, message = "Name must be at least 5 characters long")
    private String name;

    @PrimaryKeyColumn(type=PrimaryKeyType.CLUSTERED,
                      ordering=Ordering.DESCENDING)
    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    @Column("ingredients")
    private List<IngredientUDT> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(TacoUDRUtils.toIngredientUDT(ingredient));
    }
}
```

Persists to the "tacos" table

Defines the partition key

Defines the clustering key

Maps the list to the "ingredients" column

As you can see, mapping the Taco class is a bit more involved. As with Ingredient, the `@Table` annotation is used to identify tacos as the name of the table that tacos should be written to. But that's the only thing similar to Ingredient.

The `id` property is still your primary key, but it's only one of two primary key columns. More specifically, the `id` property is annotated with `@PrimaryKeyColumn` with a type of `PrimaryKeyType.PARTITIONED`. This specifies that the `id` property serves as the partition key, used to determine to which Cassandra partition(s) each row of taco data will be written.

You'll also notice that the `id` property is now a UUID instead of a Long. Although it's not required, properties that hold a generated ID value are commonly of type UUID. Moreover, the UUID is initialized with a time-based UUID value for new Taco objects (but which may be overridden when reading an existing Taco from the database).

A little further down, you see the `createdAt` property that's mapped as another primary key column. But in this case, the type attribute of `@PrimaryKeyColumn` is set to `PrimaryKeyType.CLUSTERED`, which designates the `createdAt` property as a clustering key. As mentioned earlier, clustering keys are used to determine the ordering of rows *within a partition*. More specifically, the ordering is set to descending order—therefore, within a given partition, newer rows appear first in the tacos table.

Finally, the `ingredients` property is now a List of IngredientUDT objects instead of a List of Ingredient objects. As you'll recall, Cassandra tables are highly denormalized and may contain data that's duplicated from other tables. Although the ingredient table will serve as the table of record for all available ingredients, the ingredients chosen for a taco will be duplicated in the ingredients column. Rather than simply reference one or more rows in the ingredients table, the ingredients property will contain full data for each chosen ingredient.

But why do you need to introduce a new IngredientUDT class? Why can't you just reuse the Ingredient class? Put simply, columns that contain collections of data, such as the ingredients column, must be collections of native types (integers, strings, and so on) or user-defined types.

In Cassandra, user-defined types enable you to declare table columns that are richer than simple native types. Often they're used as a denormalized analog for relational foreign keys. In contrast to foreign keys, which only hold a reference to a row in another table, columns with user-defined types actually carry data that may be copied from a row in another table. In the case of the ingredients column in the tacos table, it will contain a collection of data structures that define the ingredients themselves.

You can't use the Ingredient class as a user-defined type, because the `@Table` annotation has already mapped it as an entity for persistence in Cassandra. Therefore, you must create a new class to define how ingredients will be stored in the ingredients column of the taco table. IngredientUDT (where UDT means *user-defined type*) is the class for the job, as shown here:

```

package tacos;

import org.springframework.data.cassandra.core.mapping.UserDefinedType;

import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;

@Data
@RequiredArgsConstructor
@NoArgsConstructor(access = AccessLevel.PRIVATE, force = true)
@UserDefinedType("ingredient")
public class IngredientUDT {

    private final String name;

    private final Ingredient.Type type;

}

```

Although `IngredientUDT` looks a lot like `Ingredient`, its mapping requirements are much simpler. It's annotated with `@UserDefinedType` to identify it as a user-defined type in Cassandra. But otherwise, it's a simple class with a few properties.

You'll also note that the `IngredientUDT` class doesn't include an `id` property. Although it could include a copy of the `id` property from the source `Ingredient`, that's not necessary. In fact, the user-defined type may include any properties you wish—it doesn't need to be a one-to-one mapping with any table definition.

I realize that it might be difficult to visualize how data in a user-defined type relates to data that's persisted to a table. Figure 4.1 shows the data model for the entire Taco Cloud database, including user-defined types.

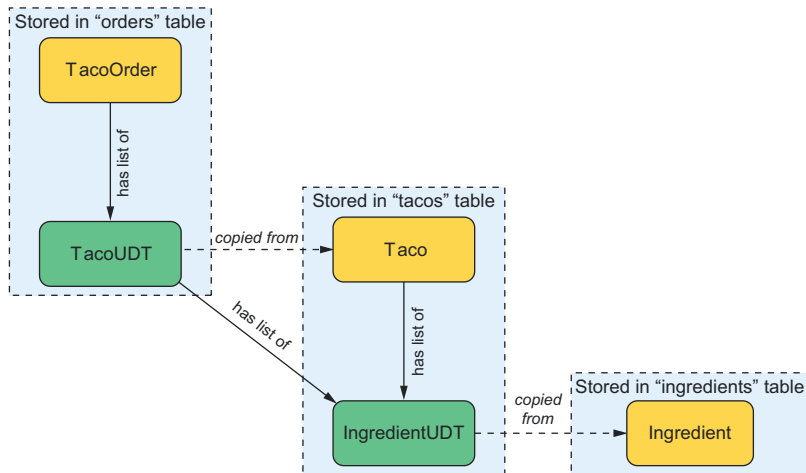


Figure 4.1 Instead of using foreign keys and joins, Cassandra tables are denormalized, with user-defined types containing data copied from related tables.

Specific to the user-defined type that you just created, notice how Taco has a list of IngredientUDT objects, which holds data copied from Ingredient objects. When a Taco is persisted, it's the Taco object and the list of IngredientUDT objects that's persisted to the tacos table. The list of IngredientUDT objects is persisted entirely within the ingredients column.

Another way of looking at this that might help you understand how user-defined types are used is to query the database for rows from the tacos table. Using CQL and the cqlsh tool that comes with Cassandra, you see the following results:

```
cqlsh:tacocloud> select id, name, createdAt, ingredients from tacos;
```

id	name	createdAt	ingredients
827390...	Carnivore	2018-04...	[{name: 'Flour Tortilla', type: 'WRAP'}, {name: 'Carnitas', type: 'PROTEIN'}, {name: 'Sour Cream', type: 'SAUCE'}, {name: 'Salsa', type: 'SAUCE'}, {name: 'Cheddar', type: 'CHEESE'}]

```
(1 rows)
```

As you can see, the id, name, and createdAt columns contain simple values. In that regard, they aren't much different than what you'd expect from a similar query against a relational database. But the ingredients column is a little different. Because it's defined as containing a collection of the user-defined ingredient type (defined by IngredientUDT), its value appears as a JSON array filled with JSON objects.

You likely noticed other user-defined types in figure 4.1. You'll certainly be creating some more as you continue mapping your domain to Cassandra tables, including some that will be used by the TacoOrder class. The next listing shows the TacoOrder class, modified for Cassandra persistence.

Listing 4.2 Mapping the TacoOrder class to a Cassandra orders table

```
package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
import java.util.UUID;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.cassandra.core.mapping.Column;
import org.springframework.data.cassandra.core.mapping.PrimaryKey;
import org.springframework.data.cassandra.core.mapping.Table;

import com.datastax.oss.driver.api.core.uuid.Uuids;
```

```

import lombok.Data;

@Data
@Table("orders")
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @PrimaryKey
    private UUID id = Uuids.timeBased();

    private Date placedAt = new Date();

    // delivery and credit card properties omitted for brevity's sake

    @Column("tacos")
    private List<TacoUDT> tacos = new ArrayList<>();

    public void addTaco(TacoUDT taco) {
        this.tacos.add(taco);
    }
}

```

Maps to the orders table

Declares the primary key

Maps a list to the tacos column

Listing 4.2 purposefully omits many of the properties of `TacoOrder` that don't lend themselves to a discussion of Cassandra data modeling. What's left are a few properties and mappings, similar to how `Taco` was defined. `@Table` is used to map `TacoOrder` to the `orders` table, much as `@Table` has been used before. In this case, you're unconcerned with ordering, so the `id` property is simply annotated with `@PrimaryKey`, designating it as both a partition key and a clustering key with default ordering.

The `tacos` property is of some interest in that it's a `List<TacoUDT>` instead of a list of `Taco` objects. The relationship between `TacoOrder` and `Taco/TacoUDT` here is similar to the relationship between `Taco` and `Ingredient/IngredientUDT`. That is, rather than joining data from several rows in a separate table through foreign keys, the `orders` table will contain all of the pertinent `taco` data, optimizing the table for quick reads.

The `TacoUDT` class is quite similar to the `IngredientUDT` class, although it does include a collection that references another user-defined type, as follows:

```

package tacos;

import java.util.List;
import org.springframework.data.cassandra.core.mapping.UserDefinedType;
import lombok.Data;

@Data
@UserDefinedType("taco")
public class TacoUDT {

    private final String name;
    private final List<IngredientUDT> ingredients;

}

```

Although it would have been nice to reuse the same domain classes you created in chapter 3, or at most to swap out some JPA annotations for Cassandra annotations, the nature of Cassandra persistence is such that it requires you to rethink how your data is modeled. But now that you’ve mapped your domain, you’re ready to write repositories.

4.1.4 Writing Cassandra repositories

As you saw in chapter 3, writing a repository with Spring Data involves simply declaring an interface that extends one of Spring Data’s base repository interfaces and optionally declaring additional query methods for custom queries. As it turns out, writing Cassandra repositories isn’t much different.

In fact, there’s very little that you’ll need to change in the repositories we’ve already written to make them work for Cassandra persistence. For example, consider the following `IngredientRepository` we created in chapter 3:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {

}
```

By extending `CrudRepository` as shown here, `IngredientRepository` is ready to persist `Ingredient` objects whose `ID` property (or, in the case of Cassandra, the primary key property) is a `String`. That’s perfect! No changes are needed for `IngredientRepository`.

The changes required for `OrderRepository` are only slightly more involved. Instead of a `Long` parameter, the `ID` parameter type specified when extending `CrudRepository` will be changed to `UUID` as follows:

```
package tacos.data;

import java.util.UUID;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, UUID> {

}
```

There’s a lot of power in Cassandra, and when it’s teamed up with Spring Data, you can wield that power in your Spring applications. But let’s shift our attention to another database for which Spring Data repository support is available: MongoDB.

4.2 Writing MongoDB repositories

MongoDB is another well-known NoSQL database. Whereas Cassandra is a column-store database, MongoDB is considered a document database. More specifically, MongoDB stores documents in BSON (Binary JSON) format, which can be queried for and retrieved in a way that's roughly similar to how you might query for data in any other database.

As with Cassandra, it's important to understand that MongoDB isn't a relational database. The way you manage your MongoDB server cluster, as well as how you model your data, requires a different mindset than when working with other kinds of databases.

That said, working with MongoDB and Spring Data isn't dramatically different from how you might use Spring Data for working with JPA or Cassandra. You'll annotate your domain classes with annotations that map the domain type to a document structure. And you'll write repository interfaces that very much follow the same programming model as those you've seen for JPA and Cassandra. Before you can do any of that, though, you must enable Spring Data MongoDB in your project.

4.2.1 Enabling Spring Data MongoDB

To get started with Spring Data MongoDB, you'll need to add the Spring Data MongoDB starter to the project build. As with Spring Data Cassandra, Spring Data MongoDB has two separate starters to choose from: one reactive and one nonreactive. We'll look at the reactive options for persistence in chapter 13. For now, add the following dependency to the build to work with the nonreactive MongoDB starter:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>
    spring-boot-starter-data-mongodb
  </artifactId>
</dependency>
```

This dependency is also available from the Spring Initializr by checking the MongoDB check box under NoSQL.

By adding the starter to the build, autoconfiguration will be triggered to enable Spring Data support for writing automatic repository interfaces, such as those you wrote for JPA in chapter 3 or for Cassandra earlier in this chapter.

By default, Spring Data MongoDB assumes that you have a MongoDB server running locally and listening on port 27017. If you have Docker installed on your machine, an easy way to get a MongoDB server running is with the following command line:

```
$ docker run -p 27017:27017 -d mongo:latest
```

But for convenience in testing or developing, you can choose to work with an embedded Mongo database instead. To do that, add the following Flapdoodle embedded MongoDB dependency to your build:

```
<dependency>
  <groupId>de.flapdoodle.embed</groupId>
  <artifactId>de.flapdoodle.embed.mongo</artifactId>
  <!-- <scope>test</scope> -->
</dependency>
```

The Flapdoodle embedded database affords you all of the same convenience of working with an in-memory Mongo database as you'd get with H2 when working with relational data. That is, you won't need to have a separate database running, but all data will be wiped clean when you restart the application.

Embedded databases are fine for development and testing, but once you take your application to production, you'll want to be sure you set a few properties to let Spring Data MongoDB know where and how your production Mongo database can be accessed, as shown next:

```
spring:
  data:
    mongodb:
      host: mongodb.tacocloud.com
      port: 27017
      username: tacocloud
      password: s3cr3tp455w0rd
      database: tacoclouddb
```

Not all of these properties are required, but they're available to help point Spring Data MongoDB in the right direction in the event that your Mongo database isn't running locally. Breaking it down, here's what each property configures:

- *spring.data.mongodb.host*—The hostname where Mongo is running (default: localhost)
- *spring.data.mongodb.port*—The port that the Mongo server is listening on (default: 27017)
- *spring.data.mongodb.username*—The username for accessing a secured Mongo database
- *spring.data.mongodb.password*—The password for accessing a secured Mongo database
- *spring.data.mongodb.database*—The database name (default: test)

Now that you have Spring Data MongoDB enabled in your project, you need to annotate your domain objects for persistence as documents in MongoDB.

4.2.2 Mapping domain types to documents

Spring Data MongoDB offers a handful of annotations that are useful for mapping domain types to document structures to be persisted in MongoDB. Although Spring Data MongoDB provides a half-dozen annotations for mapping, only the following four are useful for most common use cases:

- `@Id`—Designates a property as the document ID (from Spring Data Commons)
- `@Document`—Declares a domain type as a document to be persisted to MongoDB
- `@Field`—Specifies the field name (and, optionally, the order) for storing a property in the persisted document
- `@Transient`—Specifies that a property is not to be persisted

Of those three annotations, only the `@Id` and `@Document` annotations are strictly required. Unless you specify otherwise, properties that aren't annotated with `@Field` or `@Transient` will assume a field name equal to the property name.

Applying these annotations to the `Ingredient` class, you get the following:

```
package tacos;

import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.AccessLevel;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Data
@Document
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
public class Ingredient {

    @Id
    private String id;
    private String name;
    private Type type;

    public enum Type {
        WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
    }
}
```

As you can see, you place the `@Document` annotation at the class level to indicate that `Ingredient` is a document entity that can be written to and read from a Mongo database. By default, the collection name (the Mongo analog to a relational database table) is based on the class name, with the first letter lowercase. Because you haven't specified otherwise, `Ingredient` objects will be persisted to a collection named `ingredient`. But you can change that by setting the `collection` attribute of `@Document` as follows:

```
@Data
@AllArgsConstructor
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@Document(collection="ingredients")
public class Ingredient {
    ...
}
```

You'll also notice that the `id` property has been annotated with `@Id`. This designates the property as being the ID of the persisted document. You can use `@Id` on any property whose type is `Serializable`, including `String` and `Long`. In this case, you're already using the `String`-defined `id` property as a natural identifier, so there's no need to change it to any other type.

So far, so good. But you'll recall from earlier in this chapter that `Ingredient` was the easy domain type to map for Cassandra. The other domain types, such as `Taco`, were a bit more challenging. Let's look at how you can map the `Taco` class to see what surprises it might hold.

MongoDB's approach to document persistence lends itself very well to the domain-driven-design way of applying persistence at the aggregate root level. Documents in MongoDB tend to be defined as aggregate roots, with members of the aggregate as subdocuments.

What that means for Taco Cloud is that because `Taco` is only ever persisted as a member of the `TacoOrder`-rooted aggregate, the `Taco` class doesn't need to be annotated as a `@Document`, nor does it need an `@Id` property. The `Taco` class can remain clean of any persistence annotations, as shown here:

```
package tacos;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;

import lombok.Data;

@Data
public class Taco {

    @NotNull
    @Size(min=5, message="Name must be at least 5 characters long")
    private String name;

    private Date createdAt = new Date();

    @Size(min=1, message="You must choose at least 1 ingredient")
    private List<Ingredient> ingredients = new ArrayList<>();

    public void addIngredient(Ingredient ingredient) {
        this.ingredients.add(ingredient);
    }
}
```

The `TacoOrder` class, however, being the root of the aggregate, will need to be annotated with `@Document` and have an `@Id` property, as follows:

```

package tacos;
import java.io.Serializable;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;

import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;

import org.hibernate.validator.constraints.CreditCardNumber;
import org.springframework.data.annotation.Id;
import org.springframework.data.mongodb.core.mapping.Document;

import lombok.Data;

@Data
@Document
public class TacoOrder implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    private String id;

    private Date placedAt = new Date();

    // other properties omitted for brevity's sake

    private List<Taco> tacos = new ArrayList<>();

    public void addTaco(Taco taco) {
        this.tacos.add(taco);
    }
}

```

For brevity's sake, I've snipped out the various delivery and credit card fields. But from what's left, it's clear that all you need is `@Document` and `@Id`, as with the other domain types.

Notice, however, that the `id` property has been changed to be a `String` (as opposed to a `Long` in the JPA version or a `UUID` in the Cassandra version). As I said earlier, `@Id` can be applied to any `Serializable` type. But if you choose to use a `String` property as the ID, you get the benefit of Mongo automatically assigning a value to it when it's saved (assuming that it's null). By choosing `String`, you get a database-managed ID assignment and needn't worry about setting that property manually.

Although there are some more-advanced and unusual use cases that require additional mapping, you'll find that for most cases, `@Document` and `@Id`, along with an occasional `@Field` or `@Transient`, are sufficient for MongoDB mapping. They certainly do the job for the Taco Cloud domain types.

All that's left is to write the repository interfaces.

4.2.3 Writing MongoDB repository interfaces

Spring Data MongoDB offers automatic repository support similar to what's provided by the Spring Data JPA and Spring Data Cassandra.

You'll start by defining a repository for persisting `Ingredient` objects as documents. As before, you can write `IngredientRepository` to extend `CrudRepository`, as shown here:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.Ingredient;

public interface IngredientRepository
    extends CrudRepository<Ingredient, String> {

}
```

Wait a minute! That looks *identical* to the `IngredientRepository` interface you wrote in section 4.1 for Cassandra! Indeed, it's the same interface, with no changes. This highlights one of the benefits of extending `CrudRepository`—it's more portable across various database types and works equally well for MongoDB as for Cassandra.

Moving on to the `OrderRepository` interface, you can see in the following snippet that it's quite straightforward:

```
package tacos.data;

import org.springframework.data.repository.CrudRepository;

import tacos.TacoOrder;

public interface OrderRepository
    extends CrudRepository<TacoOrder, String> {

}
```

Just like `IngredientRepository`, `OrderRepository` extends `CrudRepository` to gain the optimizations afforded in its `insert()` methods. Otherwise, there's nothing terribly special about this repository, compared to some of the other repositories you've defined thus far. Note, however, that the ID parameter when extending `CrudRepository` is now `String` instead of `Long` (as for JPA) or `UUID` (as for Cassandra). This reflects the change we made in `TacoOrder` to support automatic assignment of IDs.

In the end, working with Spring Data MongoDB isn't drastically different from the other Spring Data projects we've worked with. The domain types are annotated differently. But aside from the ID parameter specified when extending `CrudRepository`, the repository interfaces are nearly identical.

Summary

- Spring Data supports repositories for a variety of NoSQL databases, including Cassandra, MongoDB, Neo4j, and Redis.
- The programming model for creating repositories differs very little across different underlying databases.
- Working with nonrelational databases demands an understanding of how to model data appropriately for how the database ultimately stores the data.

5

Securing Spring

This chapter covers

- Autoconfiguring Spring Security
- Defining custom user storage
- Customizing the login page
- Securing against CSRF attacks
- Knowing your user

Have you ever noticed that most people in television sitcoms don't lock their doors? In the days of *Leave It to Beaver*, it wasn't so unusual for people to leave their doors unlocked. But it seems crazy that at a time when we're concerned with privacy and security, we see television characters enabling unhindered access to their apartments and homes.

Information is probably the most valuable item we now have; crooks are looking for ways to steal our data and identities by sneaking into unsecured applications. As software developers, we must take steps to protect the information that resides in our applications. Whether it's an email account protected with a username-password pair or a brokerage account protected with a trading PIN, security is a crucial aspect of most applications.

5.1 *Enabling Spring Security*

The very first step in securing your Spring application is to add the Spring Boot security starter dependency to your build. In the project's `pom.xml` file, add the following `<dependency>` entry:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

If you're using Spring Tool Suite, this is even easier. Right-click on the `pom.xml` file and select `Edit Starters` from the Spring context menu. In the starter dependencies dialog box, select the Spring Security entry under the Security category, as shown in figure 5.1.

Believe it or not, that dependency is the only thing that's required to secure an application. When the application starts, autoconfiguration will detect that Spring Security is in the classpath and will set up some basic security configuration.

If you want to try it out, fire up the application and try to visit the home page (or any page, for that matter). You'll be prompted for authentication with a rather plain login page that looks something like figure 5.2.

TIP Going incognito: You may find it useful to set your browser to private or incognito mode when manually testing security. This will ensure that you have a fresh session each time you open a private/incognito window. You'll have to sign in to the application each time, but you can be assured that any changes you've made in security are applied and that there aren't any remnants of an older session preventing you from seeing your changes.

To get past the login page, you'll need to provide a username and password. The username is `user`. As for the password, it's randomly generated and written to the application log file. The log entry will look something like this:

```
Using generated security password: 087cfc6a-027d-44bc-95d7-cbb3a798a1ea
```

Assuming you enter the username and password correctly, you'll be granted access to the application.

It seems that securing Spring applications is pretty easy work. With the Taco Cloud application secured, I suppose I could end this chapter now and move on to the next topic. But before we get ahead of ourselves, let's consider what kind of security autoconfiguration has provided.

By doing nothing more than adding the security starter to the project build, you get the following security features:

- All HTTP request paths require authentication.
- No specific roles or authorities are required.
- Authentication is prompted with a simple login page.
- There's only one user; the username is `user`.

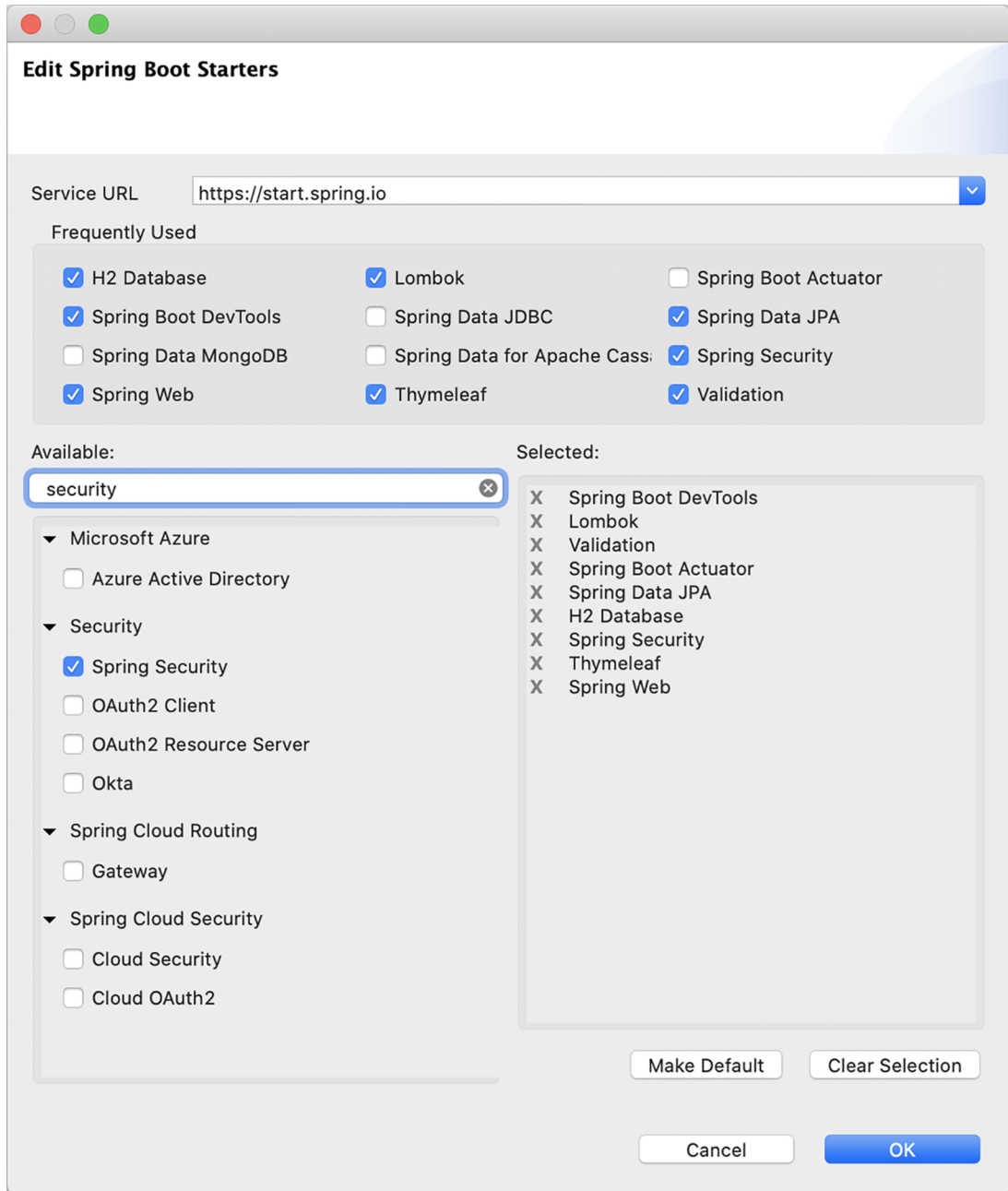


Figure 5.1 Adding the security starter with Spring Tool Suite

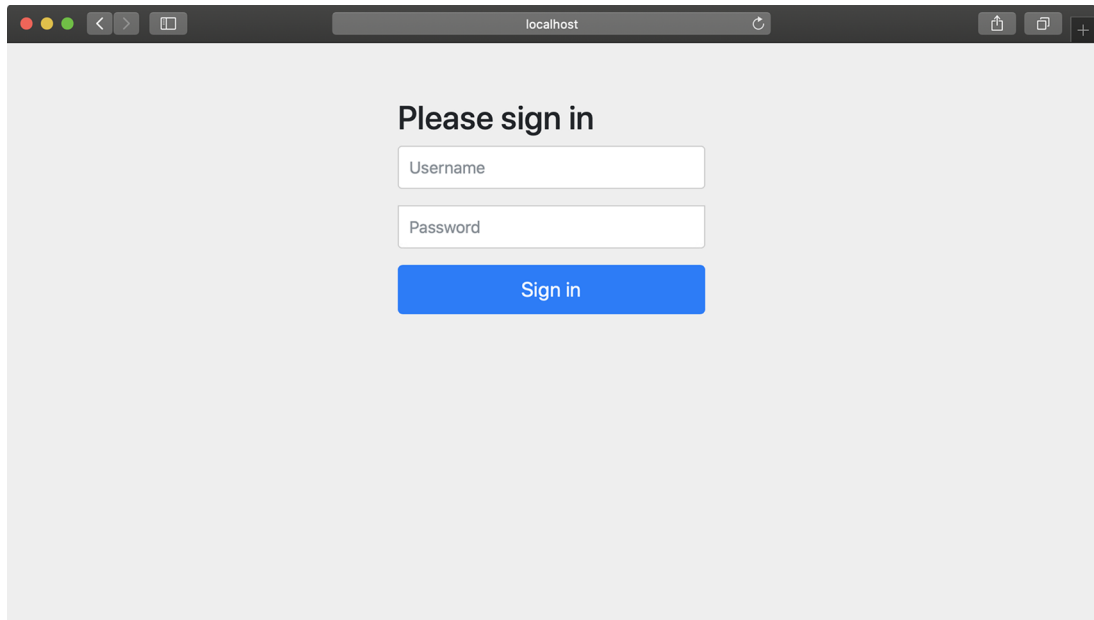


Figure 5.2 Spring Security gives you a plain login page for free.

This is a good start, but I think that the security needs of most applications (Taco Cloud included) will be quite different from these rudimentary security features.

You have more work to do if you're going to properly secure the Taco Cloud application. You'll need to at least configure Spring Security to do the following:

- Provide a login page that is designed to match the website.
- Provide for multiple users, and enable a registration page so new Taco Cloud customers can sign up.
- Apply different security rules for different request paths. The home page and registration pages, for example, shouldn't require authentication at all.

To meet your security needs for Taco Cloud, you'll have to write some explicit configuration, overriding what autoconfiguration has given you. You'll start by configuring a proper user store so that you can have more than one user.

5.2 **Configuring authentication**

Over the years, several ways of configuring Spring Security have existed, including lengthy XML configuration. Fortunately, several recent versions of Spring Security have supported Java configuration, which is much easier to read and write.

Before this chapter is finished, you'll have configured all of your Taco Cloud security needs in a Java configuration for Spring Security. But to get started, you'll ease into it by writing the configuration class shown in the following listing.

Listing 5.1 A barebones configuration class for Spring Security

```
package tacos.security;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class SecurityConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

}
```

What does this barebones security configuration do for you? Not much, actually. The main thing it does is declare a `PasswordEncoder` bean, which we'll use both when creating new users and when authenticating users at login. In this case, we're using `BCryptPasswordEncoder`, one of a handful of password encoders provided by Spring Security, including the following:

- *BCryptPasswordEncoder*—Applies bcrypt strong hashing encryption
- *NoOpPasswordEncoder*—Applies no encoding
- *Pbkdf2PasswordEncoder*—Applies PBKDF2 encryption
- *SCryptPasswordEncoder*—Applies Scrypt hashing encryption
- *StandardPasswordEncoder*—Applies SHA-256 hashing encryption

No matter which password encoder you use, it's important to understand that the password in the database is never decoded. Instead, the password that the user enters at login is encoded using the same algorithm, and it's then compared with the encoded password in the database. That comparison is performed in the `PasswordEncoder`'s `matches()` method.

Which password encoder should you use?

Not all password encoders are created equal. Ultimately, you'll need to weigh each password encoder's algorithm against your security goals and decide for yourself. But you should avoid a couple of password encoders for production applications.

`NoOpPasswordEncoder` applies no encryption whatsoever. Therefore, although it may be useful for testing, it is unsuitable for production use. And `StandardPasswordEncoder` is not considered secure enough for password encryption and has, in fact, been deprecated.

Instead, consider one of the other password encoders, any of which are more secure. We're going to use `BCryptPasswordEncoder` for the examples in this book.

In addition to the password encoder, we'll fill in this configuration class with more beans to define the specifics of security for our application. We'll start by configuring a user store that can handle more than one user.

To configure a user store for authentication purposes, you'll need to declare a `UserDetailsService` bean. The `UserDetailsService` interface is relatively simple, including only one method that must be implemented. Here's what `UserDetailsService` looks like:

```
public interface UserDetailsService {

    UserDetails loadUserByUsername(String username) throws
        UsernameNotFoundException;

}
```

The `loadUserByUsername()` method accepts a username and uses it to look up a `UserDetails` object. If no user can be found for the given username, then it will throw a `UsernameNotFoundException`.

As it turns out, Spring Security offers several out-of-the-box implementations of `UserDetailsService`, including the following:

- An in-memory user store
- A JDBC user store
- An LDAP user store

Or, you can also create your own implementation to suit your application's specific security needs.

To get started, let's try out the in-memory implementation of `UserDetailsService`.

5.2.1 *In-memory user details service*

One place where user information can be kept is in memory. Suppose you have only a handful of users, none of which are likely to change. In that case, it may be simple enough to define those users as part of the security configuration.

The following bean method shows how to create an `InMemoryUserDetailsManager` with two users, "buzz" and "woody," for that purpose.

Listing 5.2 Declaring users in an in-memory user details service bean

```
@Bean
public UserDetailsService userDetailsService>PasswordEncoder encoder) {
    List<UserDetails> userList = new ArrayList<>();
    userList.add(new User(
        "buzz", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
    userList.add(new User(
        "woody", encoder.encode("password"),
        Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"))));
    return new InMemoryUserDetailsManager(userList);
}
```

Here, a list of Spring Security User objects are created, each with a username, password, and a list of one or more authorities. Then an `InMemoryUserDetailsManager` is created using that list.

If you try out the application now, you should be able to log in as either “woody” or “buzz,” using *password* as the password.

The in-memory user details service is convenient for testing purposes or for very simple applications, but it doesn’t allow for easy editing of users. If you need to add, remove, or change a user, you’ll have to make the necessary changes and then rebuild and redeploy the application.

For the Taco Cloud application, you want customers to be able to register with the application and manage their own user accounts. That doesn’t fit with the limitations of the in-memory user details service. So let’s take a look at how to create our own implementation of `UserDetailsService` that allows for a user store database.

5.2.2 Customizing user authentication

In the previous chapter, you settled on using Spring Data JPA as your persistence option for all taco, ingredient, and order data. It would thus make sense to persist user data in the same way. If you do so, the data will ultimately reside in a relational database, so you could use JDBC authentication. But it’d be even better to leverage the Spring Data JPA repository used to store users.

First things first, though. Let’s create the domain object and repository interface that represents and persists user information.

DEFINING THE USER DOMAIN AND PERSISTENCE

When Taco Cloud customers register with the application, they’ll need to provide more than just a username and password. They’ll also give you their full name, address, and phone number. This information can be used for a variety of purposes, including pre-populating the order form (not to mention potential marketing opportunities).

To capture all of that information, you’ll create a `User` class, as follows.

Listing 5.3 Defining a user entity

```
package tacos;
import java.util.Arrays;
import java.util.Collection;
import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.
    SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import lombok.AccessLevel;
import lombok.Data;
import lombok.NoArgsConstructor;
import lombok.RequiredArgsConstructor;
```

```

@Entity
@Data
@NoArgsConstructor(access=AccessLevel.PRIVATE, force=true)
@RequiredArgsConstructor
public class User implements UserDetails {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;

    private final String username;
    private final String password;
    private final String fullname;
    private final String street;
    private final String city;
    private final String state;
    private final String zip;
    private final String phoneNumber;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        return Arrays.asList(new SimpleGrantedAuthority("ROLE_USER"));
    }

    @Override
    public boolean isAccountNonExpired() {
        return true;
    }

    @Override
    public boolean isAccountNonLocked() {
        return true;
    }

    @Override
    public boolean isCredentialsNonExpired() {
        return true;
    }

    @Override
    public boolean isEnabled() {
        return true;
    }
}

```

The first thing to notice about this `User` type is that it's not the same as the `User` class we used when creating the in-memory user details service. This one has more details about the user that we'll need to fulfill taco orders, including the user's address and contact information.

You've also probably noticed that the `User` class is a bit more involved than any of the other entities defined in chapter 3. In addition to defining a handful of properties, `User` also implements the `UserDetails` interface from Spring Security.

Implementations of `UserDetails` will provide some essential user information to the framework, such as what authorities are granted to the user and whether the user's account is enabled.

The `getAuthorities()` method should return a collection of authorities granted to the user. The various `is*` methods return a boolean to indicate whether the user's account is enabled, locked, or expired.

For your `User` entity, the `getAuthorities()` method simply returns a collection indicating that all users will have been granted `ROLE_USER` authority. And, at least for now, Taco Cloud has no need to disable users, so all of the `is*` methods return `true` to indicate that the users are active.

With the `User` entity defined, you can now define the repository interface as follows:

```
package tacos.data;
import org.springframework.data.repository.CrudRepository;
import tacos.User;

public interface UserRepository extends CrudRepository<User, Long> {

    User findByUsername(String username);

}
```

In addition to the CRUD operations provided by extending `CrudRepository`, `UserRepository` defines a `findByUsername()` method that you'll use in the user details service to look up a `User` by their username.

As you learned in chapter 3, Spring Data JPA automatically generates the implementation of this interface at run time. Therefore, you're now ready to write a custom user details service that uses this repository.

CREATING A USER DETAILS SERVICE

As you'll recall, the `UserDetailsService` interface defines only a single `loadUserByUsername()` method. That means it is a functional interface and can be implemented as a lambda instead of as a full-blown implementation class. Because all we really need is for our custom `UserDetailsService` to delegate to the `UserRepository`, it can be simply declared as a bean using the following configuration method.

Listing 5.4 Defining a custom user details service bean

```
@Bean
public UserDetailsService userDetailsService(UserRepository userRepo) {
    return username -> {
        User user = userRepo.findByUsername(username);
        if (user != null) return user;

        throw new UsernameNotFoundException("User '" + username + "' not found");
    };
}
```

The `userDetailsService()` method is given a `UserRepository` as a parameter. To create the bean, it returns a lambda that takes a `username` parameter and uses it to call `findByUsername()` on the given `UserRepository`.

The `loadByUsername()` method has one simple rule: it must never return `null`. Therefore, if the call to `findByUsername()` returns `null`, the lambda will throw a `UsernameNotFoundException` (which is defined by Spring Security). Otherwise, the `User` that was found will be returned.

Now that you have a custom user details service that reads user information via a JPA repository, you just need a way to get users into the database in the first place. You need to create a registration page for Taco Cloud patrons to register with the application.

REGISTERING USERS

Although Spring Security handles many aspects of security, it really isn't directly involved in the process of user registration, so you're going to rely on a little bit of Spring MVC to handle that task. The `RegistrationController` class in the following listing presents and processes registration forms.

Listing 5.5 A user registration controller

```
package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import tacos.data.UserRepository;

@Controller
@RequestMapping("/register")
public class RegistrationController {

    private UserRepository userRepo;
    private PasswordEncoder passwordEncoder;

    public RegistrationController(
        UserRepository userRepo, PasswordEncoder passwordEncoder) {
        this.userRepo = userRepo;
        this.passwordEncoder = passwordEncoder;
    }

    @GetMapping
    public String registerForm() {
        return "registration";
    }

    @PostMapping
    public String processRegistration(RegistrationForm form) {
        userRepo.save(form.toUser(passwordEncoder));
        return "redirect:/login";
    }
}
```

Like any typical Spring MVC controller, `RegistrationController` is annotated with `@Controller` to designate it as a controller and to mark it for component scanning. It's also annotated with `@RequestMapping` such that it will handle requests whose path is `/register`.

More specifically, a GET request for `/register` will be handled by the `registerForm()` method, which simply returns a logical view name of registration. The following listing shows a Thymeleaf template that defines the registration view.

Listing 5.6 A Thymeleaf registration form view

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Register</h1>

    <form method="POST" th:action="@{/register}" id="registerForm">

      <label for="username">Username: </label>
      <input type="text" name="username"/><br/>

      <label for="password">Password: </label>
      <input type="password" name="password"/><br/>

      <label for="confirm">Confirm password: </label>
      <input type="password" name="confirm"/><br/>

      <label for="fullname">Full name: </label>
      <input type="text" name="fullname"/><br/>

      <label for="street">Street: </label>
      <input type="text" name="street"/><br/>

      <label for="city">City: </label>
      <input type="text" name="city"/><br/>

      <label for="state">State: </label>
      <input type="text" name="state"/><br/>

      <label for="zip">Zip: </label>
      <input type="text" name="zip"/><br/>

      <label for="phone">Phone: </label>
      <input type="text" name="phone"/><br/>

      <input type="submit" value="Register"/>
    </form>
```

```

    </body>
</html>

```

When the form is submitted, the `processRegistration()` method handles the HTTPS POST request. The form fields will be bound to a `RegistrationForm` object by Spring MVC and passed into the `processRegistration()` method for processing. `RegistrationForm` is defined in the following class:

```

package tacos.security;
import org.springframework.security.crypto.password.PasswordEncoder;
import lombok.Data;
import tacos.User;

@Data
public class RegistrationForm {

    private String username;
    private String password;
    private String fullname;
    private String street;
    private String city;
    private String state;
    private String zip;
    private String phone;

    public User toUser(PasswordEncoder passwordEncoder) {
        return new User(
            username, passwordEncoder.encode(password),
            fullname, street, city, state, zip, phone);
    }
}

```

For the most part, `RegistrationForm` is just a basic Lombok class with a handful of properties. But the `toUser()` method uses those properties to create a new `User` object, which is what `processRegistration()` will save, using the injected `UserRepository`.

You’ve no doubt noticed that `RegistrationController` is injected with a `PasswordEncoder`. This is the exact same `PasswordEncoder` bean you declared earlier. When processing a form submission, `RegistrationController` passes it to the `toUser()` method, which uses it to encode the password before saving it to the database. In this way, the submitted password is written in an encoded form, and the user details service will be able to authenticate against that encoded password.

Now the Taco Cloud application has complete user registration and authentication support. But if you start it up at this point, you’ll notice that you can’t even get to the registration page without being prompted to log in. That’s because, by default, all requests require authentication. Let’s look at how web requests are intercepted and secured so you can fix this strange chicken-and-egg situation.

5.3 Securing web requests

The security requirements for Taco Cloud should require that a user be authenticated before designing tacos or placing orders. But the home page, login page, and registration page should be available to unauthenticated users.

To configure these security rules, we'll need to declare a `SecurityFilterChain` bean. The following `@Bean` method shows a minimal (but not useful) `SecurityFilterChain` bean declaration:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http.build();
}
```

The `filterChain()` method accepts an `HttpSecurity` object, which acts as a builder that can be used to configure how security is handled at the web level. Once security configuration is set up via the `HttpSecurity` object, a call to `build()` will create a `SecurityFilterChain` that is returned from the bean method.

The following are among the many things you can configure with `HttpSecurity`:

- Requiring that certain security conditions be met before allowing a request to be served
- Configuring a custom login page
- Enabling users to log out of the application
- Configuring cross-site request forgery protection

Intercepting requests to ensure that the user has proper authority is one of the most common things you'll configure `HttpSecurity` to do. Let's ensure that your Taco Cloud customers meet those requirements.

5.3.1 Securing requests

You need to ensure that requests for `/design` and `/orders` are available only to authenticated users; all other requests should be permitted for all users. The following configuration does exactly that:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").hasRole("USER")
            .antMatchers("/", "**").permitAll()

        .and()
        .build();
}
```

The call to `authorizeRequests()` returns an object (`ExpressionUrlAuthorizationConfigurer.ExpressionInterceptUrlRegistry`) on which you can specify URL paths

and patterns and the security requirements for those paths. In this case, you specify the following two security rules:

- Requests for `/design` and `/orders` should be for users with a granted authority of `ROLE_USER`. Don't include the `ROLE_` prefix on roles passed to `hasRole()`; it will be assumed by `hasRole()`.
- All requests should be permitted to all users.

The order of these rules is important. Security rules declared first take precedence over those declared lower down. If you were to swap the order of those two security rules, all requests would have `permitAll()` applied to them; the rule for `/design` and `/orders` requests would have no effect.

The `hasRole()` and `permitAll()` methods are just a couple of the methods for declaring security requirements for request paths. Table 5.1 describes all the available methods.

Table 5.1 Configuration methods to define how a path is to be secured

Method	What it does
<code>access(String)</code>	Allows access if the given Spring Expression Language (SpEL) expression evaluates to <code>true</code>
<code>anonymous()</code>	Allows access to anonymous users
<code>authenticated()</code>	Allows access to authenticated users
<code>denyAll()</code>	Denies access unconditionally
<code>fullyAuthenticated()</code>	Allows access if the user is fully authenticated (not remembered)
<code>hasAnyAuthority(String...)</code>	Allows access if the user has any of the given authorities
<code>hasAnyRole(String...)</code>	Allows access if the user has any of the given roles
<code>hasAuthority(String)</code>	Allows access if the user has the given authority
<code>hasIpAddress(String)</code>	Allows access if the request comes from the given IP address
<code>hasRole(String)</code>	Allows access if the user has the given role
<code>not()</code>	Negates the effect of any of the other access methods
<code>permitAll()</code>	Allows access unconditionally
<code>rememberMe()</code>	Allows access for users who are authenticated via <code>remember-me</code>

Most of the methods in table 5.1 provide essential security rules for request handling, but they're self-limiting, enabling security rules only as defined by those methods. Alternatively, you can use the `access()` method to provide a SpEL expression to declare richer security rules. Spring Security extends SpEL to include several security-specific values and functions, as listed in table 5.2.

Table 5.2 Spring Security extensions to the Spring Expression Language

Security expression	What it evaluates to
<code>authentication</code>	The user's authentication object
<code>denyAll</code>	Always evaluates to <code>false</code>
<code>hasAnyAuthority(String... authorities)</code>	true if the user has been granted any of the given authorities
<code>hasAnyRole(String... roles)</code>	true if the user has any of the given roles
<code>hasAuthority(String authority)</code>	true if the user has been granted the specified authority
<code>hasPermission(Object target, Object permission)</code>	true if the user has access to the specified target object for the given permission
<code>hasPermission(Serializable targetId, String targetType, Object permission)</code>	true if the user has access to the object specified by <code>targetId</code> and the specified <code>targetType</code> for the given permission
<code>hasRole(String role)</code>	true if the user has the given role
<code>hasIpAddress(String ipAddress)</code>	true if the request comes from the given IP address
<code>isAnonymous()</code>	true if the user is anonymous
<code>isAuthenticated()</code>	true if the user is authenticated
<code>isFullyAuthenticated()</code>	true if the user is fully authenticated (not authenticated with <code>remember-me</code>)
<code>isRememberMe()</code>	true if the user is authenticated via <code>remember-me</code>
<code>permitAll</code>	Always evaluates to <code>true</code>
<code>principal</code>	The user's principal object

As you can see, most of the security expression extensions in table 5.2 correspond to similar methods in table 5.1. In fact, using the `access()` method along with the `hasRole()` and `permitAll` expressions, you can rewrite the `SecurityFilterChain` configuration as follows.

Listing 5.7 Using Spring expressions to define authorization rules

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").access("hasRole('USER')")
            .antMatchers("/", "/*").access("permitAll()")

        .and()
        .build();
}
```

This may not seem like a big deal at first. After all, these expressions only mirror what you already did with method calls. But expressions can be much more flexible. For instance, suppose that (for some crazy reason) you wanted to allow only users with `ROLE_USER` authority to create new tacos on Tuesdays (for example, on Taco Tuesday); you could rewrite the expression as shown in this modified version of the `SecurityFilterChain` bean method:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders")
                .access("hasRole('USER') && " +
                    "T(java.util.Calendar).getInstance().get(" +
                    "T(java.util.Calendar).DAY_OF_WEEK) == " +
                    "T(java.util.Calendar).TUESDAY")
                .antMatchers("/", "**").access("permitAll")
            .and()
        .build();
}
```

With SpEL security constraints, the possibilities are virtually endless. I'll bet that you're already dreaming up interesting security constraints based on SpEL.

The authorization needs for the Taco Cloud application are met by the simple use of `access()` and the SpEL expressions. Now let's see about customizing the login page to fit the look of the Taco Cloud application.

5.3.2 *Creating a custom login page*

The default login page is much better than the clunky HTTP basic dialog box you started with, but it's still rather plain and doesn't quite fit with the look of the rest of the Taco Cloud application.

To replace the built-in login page, you first need to tell Spring Security what path your custom login page will be at. That can be done by calling `formLogin()` on the `HttpSecurity` object, as shown next:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .antMatchers("/design", "/orders").access("hasRole('USER')")
            .antMatchers("/", "**").access("permitAll()")
        .and()
        .formLogin()
            .loginPage("/login")
        .and()
        .build();
}
```

Notice that before you call `formLogin()`, you bridge this section of configuration and the previous section with a call to `and()`. The `and()` method signifies that you're finished with the authorization configuration and are ready to apply some additional HTTP configuration. You'll use `and()` several times as you begin new sections of configuration.

After the bridge, you call `formLogin()` to start configuring your custom login form. The call to `loginPage()` after that designates the path where your custom login page will be provided. When Spring Security determines that the user is unauthenticated and needs to log in, it will redirect them to this path.

Now you need to provide a controller that handles requests at that path. Because your login page will be fairly simple—nothing but a view—it's easy enough to declare it as a view controller in `WebConfig`. The following `addViewControllers()` method sets up the login page view controller alongside the view controller that maps `/` to the home controller:

```
@Override
public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("home");
    registry.addViewController("/login");
}
```

Finally, you need to define the login page view itself. Because you're using Thymeleaf as your template engine, the following Thymeleaf template should do fine:

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
  </head>

  <body>
    <h1>Login</h1>
    

    <div th:if="${error}">
      Unable to login. Check your username and password.
    </div>

    <p>New here? Click
      <a th:href="@{/register}">here</a> to register.</p>

    <form method="POST" th:action="@{/login}" id="loginForm">
      <label for="username">Username: </label>
      <input type="text" name="username" id="username" /><br/>

      <label for="password">Password: </label>
      <input type="password" name="password" id="password" /><br/>

      <input type="submit" value="Login"/>
    </form>
```

```
</body>  
</html>
```

The key things to note about this login page are the path it posts to and the names of the username and password fields. By default, Spring Security listens for login requests at `/login` and expects that the username and password fields be named `username` and `password`. This is configurable, however. For example, the following configuration customizes the path and field names:

```
.and()  
  .formLogin()  
    .loginPage("/login")  
    .loginProcessingUrl("/authenticate")  
    .usernameParameter("user")  
    .passwordParameter("pwd")
```

Here, you specify that Spring Security should listen for requests to `/authenticate` to handle login submissions. Also, the username and password fields should now be named `user` and `pwd`.

By default, a successful login will take the user directly to the page that they were navigating to when Spring Security determined that they needed to log in. If the user were to directly navigate to the login page, a successful login would take them to the root path (for example, the home page). But you can change that by specifying a default success page, as shown next:

```
.and()  
  .formLogin()  
    .loginPage("/login")  
    .defaultSuccessUrl("/design")
```

As configured here, if the user were to successfully log in after directly going to the login page, they would be directed to the `/design` page.

Optionally, you can force the user to the design page after login, even if they were navigating elsewhere prior to logging in, by passing `true` as a second parameter to `defaultSuccessUrl` as follows:

```
.and()  
  .formLogin()  
    .loginPage("/login")  
    .defaultSuccessUrl("/design", true)
```

Signing in with a username and password is the most common way to authenticate in a web application. But let's have a look at another way to authenticate a user that uses someone else's login page.

5.3.3 Enabling third-party authentication

You may have seen links or buttons on your favorite website that say “Sign in with Facebook,” “Log in with Twitter,” or something similar. Rather than asking a user to enter their credentials on a login page specific to the website, they offer a way to sign in via another website like Facebook that they may already be logged into.

This type of authentication is based on OAuth2 or OpenID Connect (OIDC). Although OAuth2 is an authorization specification, and we’ll talk more about how to use it to secure REST APIs in chapter 8, it can be also used to perform authentication via a third-party website. OpenID Connect is another security specification that is based on OAuth2 to formalize the interaction that takes place during a third-party authentication.

To employ this type of authentication in your Spring application, you’ll need to add the OAuth2 client starter to the build as follows:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
```

Then, at the very least, you’ll need to configure details about one or more OAuth2 or OpenID Connect servers that you want to be able to authenticate against. Spring Security supports sign-in with Facebook, Google, GitHub, and Okta out of the box, but you can configure other clients by specifying a few extra properties.

The general set of properties you’ll need to set for your application to act as an OAuth2/OpenID Connect client follows:

```
spring:
  security:
    oauth2:
      client:
        registration:
          <oauth2 or openid provider name>:
            clientId: <client id>
            clientSecret: <client secret>
            scope: <comma-separated list of requested scopes>
```

For example, suppose that for Taco Cloud, we want users to be able to sign in using Facebook. The following configuration in `application.yml` will set up the OAuth2 client:

```
spring:
  security:
    oauth2:
      client:
        registration:
          facebook:
            clientId: <facebook client id>
            clientSecret: <facebook client secret>
            scope: email, public_profile
```

The client ID and secret are the credentials that identify your application to Facebook. You can obtain a client ID and secret by creating a new application entry at <https://developers.facebook.com/>. The scope property specifies the access that the application will be granted. In this case, the application will have access to the user's email address and the essential information from their public Facebook profile.

In a very simple application, this is all you will need. When the user attempts to access a page that requires authentication, their browser will redirect to Facebook. If they're not already logged in to Facebook, they'll be greeted with the Facebook sign-in page. After signing in to Facebook, they'll be asked to authorize your application and grant the requested scope. Finally, they'll be redirected back to your application, where they will have been authenticated.

If, however, you've customized security by declaring a `SecurityFilterChain` bean, then you'll need to enable OAuth2 login along with the rest of the security configuration as follows:

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeRequests()
            .mvcMatchers("/design", "/orders").hasRole("USER")
            .anyRequest().permitAll()

        .and()
            .formLogin()
                .loginPage("/login")

        .and()
            .oauth2Login()

        ...

        .and()
            .build();
}
```

You may also want to offer both a traditional username-password login and third-party login. In that case, you can specify the login page in the configuration like this:

```
.and()
    .oauth2Login()
        .loginPage("/login")
```

This will cause the application to always take the user to the application-provided login page where they may choose to log in with their username and password as usual. But you can also provide a link on that same login page that offers them the opportunity to log in with Facebook. Such a link could look like this in the login page's HTML template:

```
<a th:href="/oauth2/authorization/facebook">Sign in with Facebook</a>
```

Now that you've dealt with logging in, let's flip to the other side of the authentication coin and see how you can enable a user to log out. Just as important as logging in to an application is logging out. To enable logout, you simply need to call `logout` on the `HttpSecurity` object as follows:

```
.and()  
  .logout()
```

This sets up a security filter that intercepts POST requests to `/logout`. Therefore, to provide logout capability, you just need to add a logout form and button to the views in your application, as shown next:

```
<form method="POST" th:action="@{/logout}">  
  <input type="submit" value="Logout"/>  
</form>
```

When the user clicks the button, their session will be cleared, and they will be logged out of the application. By default, they'll be redirected to the login page where they can log in again. But if you'd rather they be sent to a different page, you can call `logoutSuccessUrl()` to specify a different post-logout landing page, as shown here:

```
.and()  
  .logout()  
    .logoutSuccessUrl("/")
```

In this case, users will be sent to the home page following logout.

5.3.4 Preventing cross-site request forgery

Cross-site request forgery (CSRF) is a common security attack. It involves subjecting a user to code on a maliciously designed web page that automatically (and usually secretly) submits a form to another application on behalf of a user who is often the victim of the attack. For example, a user may be presented with a form on an attacker's website that automatically posts to a URL on the user's banking website (which is presumably poorly designed and vulnerable to such an attack) to transfer money. The user may not even know that the attack happened until they notice money missing from their account.

To protect against such attacks, applications can generate a CSRF token upon displaying a form, place that token in a hidden field, and then stow it for later use on the server. When the form is submitted, the token is sent back to the server along with the rest of the form data. The request is then intercepted by the server and compared with the token that was originally generated. If the token matches, the request is allowed to proceed. Otherwise, the form must have been rendered by an evil website without knowledge of the token generated by the server.

Fortunately, Spring Security has built-in CSRF protection. Even more fortunate is that it's enabled by default and you don't need to explicitly configure it. You only

need to make sure that any forms your application submits include a field named `_csrf` that contains the CSRF token.

Spring Security even makes that easy by placing the CSRF token in a request attribute with the name `_csrf`. Therefore, you could render the CSRF token in a hidden field with the following in a Thymeleaf template:

```
<input type="hidden" name="_csrf" th:value="${_csrf.token}"/>
```

If you're using Spring MVC's JSP tag library or Thymeleaf with the Spring Security dialect, you needn't even bother explicitly including a hidden field. The hidden field will be rendered automatically for you.

In Thymeleaf, you just need to make sure that one of the attributes of the `<form>` element is prefixed as a Thymeleaf attribute. That's usually not a concern, because it's quite common to let Thymeleaf render the path as context relative. For example, the `th:action` attribute shown next is all you need for Thymeleaf to render the hidden field for you:

```
<form method="POST" th:action="@{/login}" id="loginForm">
```

It's possible to disable CSRF support, but I'm hesitant to show you how. CSRF protection is important and easily handled in forms, so there's little reason to disable it. But if you insist on disabling it, you can do so by calling `disable()` like this:

```
.and()  
  .csrf()  
    .disable()
```

Again, I caution you not to disable CSRF protection, especially for production applications.

All of your web layer security is now configured for Taco Cloud. Among other things, you now have a custom login page and the ability to authenticate users against a JPA user repository. Now let's see how you can obtain information about the logged-in user.

5.4 *Applying method-level security*

Although it's easy to think about security at the web-request level, that's not always where security constraints are best applied. Sometimes it's better to verify that the user is authenticated and has been granted adequate authority at the point where the secured action will be performed.

For example, let's say that for administrative purposes, we have a service class that includes a method for clearing out all orders from the database. Using an injected `OrderRepository`, that method might look something like this:

```
public void deleteAllOrders() {  
    orderRepository.deleteAll();  
}
```

Now, suppose we have a controller that calls the `deleteAllOrders()` method as the result of a POST request, as shown here:

```
@Controller
@RequestMapping("/admin")
public class AdminController {

    private OrderAdminService adminService;

    public AdminController(OrderAdminService adminService) {
        this.adminService = adminService;
    }

    @PostMapping("/deleteOrders")
    public String deleteAllOrders() {
        adminService.deleteAllOrders();
        return "redirect:/admin";
    }
}
```

It'd be easy enough to tweak `SecurityConfig` as follows to ensure that only authorized users are allowed to perform that POST request:

```
.authorizeRequests()
...
.antMatchers(HttpMethod.POST, "/admin/**")
    .access("hasRole('ADMIN')")
....
```

That's great and would prevent any unauthorized user from making a POST request to `/admin/deleteOrders` that would result in all orders disappearing from the database.

But suppose that some other controller method also calls `deleteAllOrders()`. You'd need to add more matchers to secure the requests for the other controllers that will need to be secured.

Instead, we can apply security directly on the `deleteAllOrders()` method like this:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAllOrders() {
    orderRepository.deleteAll();
}
```

The `@PreAuthorize` annotation takes a SpEL expression, and, if the expression evaluates to false, the method will not be invoked. On the other hand, if the expression evaluates to true, then the method will be allowed. In this case, `@PreAuthorize` is checking that the user has the `ROLE_ADMIN` privilege. If so, then the method will be called and all orders will be deleted. Otherwise, it will be stopped in its tracks.

In the event that `@PreAuthorize` blocks the call, then Spring Security's `AccessDeniedException` will be thrown. This is an unchecked exception, so you don't need

to catch it, unless you want to apply some custom behavior around the exception handling. If left uncaught, it will bubble up and eventually be caught by Spring Security's filters and handled accordingly, either with an HTTP 403 page or perhaps by redirecting to the login page if the user is unauthenticated.

For `@PreAuthorize` to work, you'll need to enable global method security. For that, you'll need to annotate the security configuration class with `@EnableGlobalMethodSecurity` as follows:

```
@Configuration
@EnableGlobalMethodSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter {
    ...
}
```

You'll find `@PreAuthorize` to be a useful annotation for most method-level security needs. But know that it has a slightly less useful after-invocation counterpart in `@PostAuthorize`. The `@PostAuthorize` annotation works almost the same as the `@PreAuthorize` annotation, except that its expression won't be evaluated until after the target method is invoked and returns. This allows the expression to consider the return value of the method in deciding whether to permit the method invocation.

For example, suppose we have a method that fetches an order by its ID. If you want to restrict it from being used except by admins or by the user who the order belongs to, you can use `@PostAuthorize` like this:

```
@PostAuthorize("hasRole('ADMIN') || " +
    "returnObject.user.username == authentication.name")
public TacoOrder getOrder(long id) {
    ...
}
```

In this case, the `returnObject` in the `TacoOrder` returned from the method. If its `user` property has a `username` that is equal to the `authentication`'s `name` property, then it will be allowed. To know that, though, the method will need to be executed so that it can return the `TacoOrder` object for consideration.

But wait! How can you secure a method from being invoked if the condition for applying security relies on the return value from the method invocation? That chicken-and-egg riddle is solved by allowing the method to be invoked, then throwing an `AccessDeniedException` if the expression returns `false`.

5.5 *Knowing your user*

Often, it's not enough to simply know that the user has logged in and what permissions they have been granted. It's usually important to also know who they are, so that you can tailor their experience.

For example, in `OrderController`, when you initially create the `TacoOrder` object that's bound to the order form, it'd be nice if you could prepopulate the `TacoOrder`

with the user's name and address, so they don't have to reenter it for each order. Perhaps even more important, when you save their order, you should associate the TacoOrder entity with the User that created the order.

To achieve the desired connection between an TacoOrder entity and a User entity, you need to add the following new property to the TacoOrder class:

```
@Data
@Entity
@Table(name="Taco_Order")
public class TacoOrder implements Serializable {

    ...

    @ManyToOne
    private User user;

    ...

}
```

The @ManyToOne annotation on this property indicates that an order belongs to a single user and, conversely, that a user may have many orders. (Because you're using Lombok, you won't need to explicitly define accessor methods for the property.)

In OrderController, the processOrder() method is responsible for saving an order. It will need to be modified to determine who the authenticated user is and to call setUser() on the TacoOrder object to connect the order with the user.

We have several ways to determine who the user is. A few of the most common ways follow:

- Inject a java.security.Principal object into the controller method.
- Inject an org.springframework.security.core.Authentication object into the controller method.
- Use org.springframework.security.core.context.SecurityContextHolder to get at the security context.
- Inject an @AuthenticationPrincipal annotated method parameter. (@AuthenticationPrincipal is from Spring Security's org.springframework.security.core.annotation package.)

For example, you could modify processOrder() to accept a java.security.Principal as a parameter. You could then use the principal name to look up the user from a UserRepository as follows:

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Principal principal) {

    ...

}
```

```

    User user = userRepository.findByUsername(
        principal.getName());

    order.setUser(user);

    ...

}

```

This works fine, but it litters code that's otherwise unrelated to security with security code. You can trim down some of the security-specific code by modifying `processOrder()` to accept an `Authentication` object as a parameter instead of a `Principal`, as shown next:

```

@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    Authentication authentication) {

    ...

    User user = (User) authentication.getPrincipal();
    order.setUser(user);

    ...

}

```

With the `Authentication` in hand, you can call `getPrincipal()` to get the principal object which, in this case, is a `User`. Note that `getPrincipal()` returns a `java.util.Object`, so you need to cast it to `User`.

Perhaps the cleanest solution of all, however, is to simply accept a `User` object in `processOrder()` but annotate it with `@AuthenticationPrincipal` so that it will be the authentication's principal, as follows:

```

@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
    SessionStatus sessionStatus,
    @AuthenticationPrincipal User user) {

    if (errors.hasErrors()) {
        return "orderForm";
    }

    order.setUser(user);

    orderRepo.save(order);
    sessionStatus.setComplete();

    return "redirect:/";
}

```

What's nice about `@AuthenticationPrincipal` is that it doesn't require a cast (as with `Authentication`), and it limits the security-specific code to the annotation itself. By the time you get the `User` object in `processOrder()`, it's ready to be used and assigned to the `TacoOrder`.

There's one other way of identifying who the authenticated user is, although it's a bit messy in the sense that it's very heavy with security-specific code. You can obtain an `Authentication` object from the security context and then request its principal like this:

```
Authentication authentication =
    SecurityContextHolder.getContext().getAuthentication();
User user = (User) authentication.getPrincipal();
```

Although this snippet is thick with security-specific code, it has one advantage over the other approaches described: it can be used anywhere in the application, not just in a controller's handler methods. This makes it suitable for use in lower levels of the code.

Summary

- Spring Security autoconfiguration is a great way to get started with security, but most applications will need to explicitly configure security to meet their unique security requirements.
- User details can be managed in user stores backed by relational databases, LDAP, or completely custom implementations.
- Spring Security automatically protects against CSRF attacks.
- Information about the authenticated user can be obtained via the `SecurityContext` object (returned from `SecurityContextHolder.getContext()`) or injected into controllers using `@AuthenticationPrincipal`.

Working with configuration properties

This chapter covers

- Fine-tuning autoconfigured beans
- Applying configuration properties to application components
- Working with Spring profiles

Do you remember when the iPhone first came out? A small slab of metal and glass hardly fit the description of what the world had come to recognize as a phone. And yet, it pioneered the modern smartphone era, changing everything about how we communicate. Although touch phones are in many ways easier and more powerful than their predecessor, the flip phone, when the iPhone was first announced, it was hard to imagine how a device with a single button could be used to place calls.

In some ways, Spring Boot autoconfiguration is like this. Autoconfiguration greatly simplifies Spring application development. But after a decade of setting property values in Spring XML configuration and calling setter methods on bean instances, it's not immediately apparent how to set properties on beans for which there's no explicit configuration.

Fortunately, Spring Boot provides a way to set property values on application components with configuration properties. Configuration properties are nothing more than properties on `@ConfigurationProperties`-annotated beans in the Spring

application context. Spring will inject values from one of several property sources—including JVM system properties, command-line arguments, and environment variables—into the bean properties. We'll see how to use `@ConfigurationProperties` on our own beans in section 6.2. But Spring Boot itself provides several `@ConfigurationProperties`-annotated beans that we'll configure first.

In this chapter, you're going to take a step back from implementing new features in the Taco Cloud application to explore configuration properties. What you take away will no doubt prove useful as you move forward in the chapters that follow. We'll start by seeing how to employ configuration properties to fine-tune what Spring Boot automatically configures.

6.1 *Fine-tuning autoconfiguration*

Before we dive in too deeply with configuration properties, it's important to establish the following different (but related) kinds of configurations in Spring:

- *Bean wiring*—Configuration that declares application components to be created as beans in the Spring application context and how they should be injected into each other
- *Property injection*—Configuration that sets values on beans in the Spring application context

In Spring's XML and Java configuration, these two types of configurations are often declared explicitly in the same place. In Java configuration, a `@Bean`-annotated method is likely to both instantiate a bean and then set values to its properties. For example, consider the following `@Bean` method that declares a `DataSource` for an embedded H2 database:

```
@Bean
public DataSource dataSource() {
    return new EmbeddedDatabaseBuilder()
        .setType(H2)
        .addScript("taco_schema.sql")
        .addScripts("user_data.sql", "ingredient_data.sql")
        .build();
}
```

Here the `addScript()` and `addScripts()` methods set some `String` properties with the name of SQL scripts that should be applied to the database once the data source is ready. Whereas this is how you might configure a `DataSource` bean if you aren't using Spring Boot, autoconfiguration makes this method completely unnecessary.

If the H2 dependency is available in the runtime classpath, then Spring Boot automatically creates in the Spring application context an appropriate `DataSource` bean, which applies the SQL scripts `schema.sql` and `data.sql`.

But what if you want to name the SQL scripts something else? Or what if you need to specify more than two SQL scripts? That's where configuration properties come in.

But before you can start using configuration properties, you need to understand where those properties come from.

6.1.1 Understanding Spring's environment abstraction

The Spring environment abstraction is a one-stop shop for any configurable property. It abstracts the origins of properties so that beans needing those properties can consume them from Spring itself. The Spring environment pulls from several property sources, including the following:

- JVM system properties
- Operating system environment variables
- Command-line arguments
- Application property configuration files

It then aggregates those properties into a single source from which Spring beans can be injected. Figure 6.1 illustrates how properties from property sources flow through the Spring environment abstraction to Spring beans.

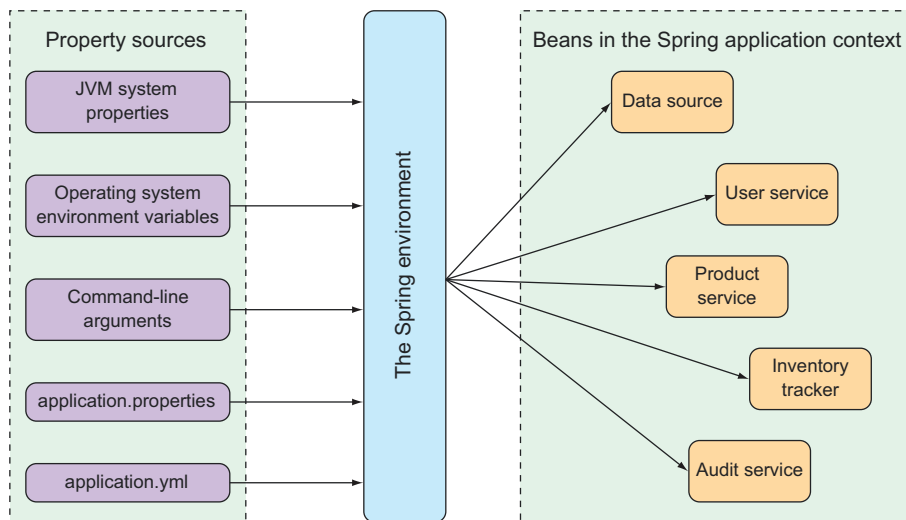


Figure 6.1 The Spring environment pulls properties from property sources and makes them available to beans in the application context.

The beans that are automatically configured by Spring Boot are all configurable by properties drawn from the Spring environment. As a simple example, suppose that you would like the application's underlying servlet container to listen for requests on some port other than the default port of 8080. To do that, specify a different port by setting the `server.port` property in `src/main/resources/application.properties` like this:

```
server.port=9090
```

Personally, I prefer using YAML when setting configuration properties. Therefore, instead of using `application.properties`, I might set the `server.port` value in `src/main/resources/application.yml` like this:

```
server:
  port: 9090
```

If you'd prefer to configure that property externally, you could also specify the port when starting the application using a command-line argument as follows:

```
$ java -jar tacocloud-0.0.5-SNAPSHOT.jar --server.port=9090
```

If you want the application to always start on a specific port, you could set it one time as an operating system environment variable, as shown next:

```
$ export SERVER_PORT=9090
```

Notice that when setting properties as environment variables, the naming style is slightly different to accommodate restrictions placed on environment variable names by the operating system. That's OK. Spring is able to sort it out and interpret `SERVER_PORT` as `server.port` with no problems.

As I said, we have several ways of setting configuration properties. In fact, you could use one of several hundred configuration properties to tweak and adjust how Spring beans behave. You've already seen a few: `server.port` in this chapter, as well as `spring.datasource.name` and `spring.thymeleaf.cache` in earlier chapters.

It's impossible to examine all of the available configuration properties in this chapter. Even so, let's take a look at a few of the most useful configuration properties you might commonly encounter. We'll start with a few properties that let you tweak the autoconfigured data source.

6.1.2 Configuring a data source

At this point, the Taco Cloud application is still unfinished, but you'll have several more chapters to take care of that before you're ready to deploy the application. As such, the embedded H2 database you're using as a data source is perfect for your needs—for now. But once you take the application into production, you'll probably want to consider a more permanent database solution.

Although you could explicitly configure your own `DataSource` bean, that's usually unnecessary. Instead, it's simpler to configure the URL and credentials for your database via configuration properties. For example, if you were to start using a MySQL database, you might add the following configuration properties to `application.yml`:

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
```

Although you'll need to add the appropriate JDBC driver to the build, you won't usually need to specify the JDBC driver class—Spring Boot can figure it out from the structure of the database URL. But if there's a problem, you can try setting the `spring.datasource.driver-class-name` property like so:

```
spring:
  datasource:
    url: jdbc:mysql://localhost/tacocloud
    username: tacouser
    password: tacopassword
    driver-class-name: com.mysql.jdbc.Driver
```

Spring Boot uses this connection data when autoconfiguring the `DataSource` bean. The `DataSource` bean will be pooled using the HikariCP connection pool if it's available on the classpath. If not, Spring Boot looks for and uses one of the following other connection pool implementations on the classpath:

- Tomcat JDBC Connection Pool
- Apache Commons DBCP2

Although these are the only connection pool options available through autoconfiguration, you're always welcome to explicitly configure a `DataSource` bean to use whatever connection pool implementation you'd like.

Earlier in this chapter, we suggested that there might be a way to specify the database initialization scripts to run when the application starts. In that case, the `spring.datasource.schema` and `spring.datasource.data` properties, shown here, prove useful:

```
spring:
  datasource:
    schema:
      - order-schema.sql
      - ingredient-schema.sql
      - taco-schema.sql
      - user-schema.sql
    data:
      - ingredients.sql
```

Maybe explicit data source configuration isn't your style. Instead, perhaps you'd prefer to configure your data source in the Java Naming and Directory Interface (JNDI) (<http://mng.bz/MvEo>) and have Spring look it up from there. In that case, set up your data source by configuring `spring.datasource.jndi-name` as follows:

```
spring:
  datasource:
    jndi-name: java:/comp/env/jdbc/tacoCloudDS
```

If you set the `spring.datasource.jndi-name` property, the other data source connection properties (if set) are ignored.

6.1.3 Configuring the embedded server

You’ve already seen how to set the servlet container’s port by setting `server.port`. What I didn’t show you is what happens if `server.port` is set to 0, as shown here:

```
server:
  port: 0
```

Although you’re explicitly setting `server.port` to 0, the server won’t start on port 0. Instead, it’ll start on a randomly chosen available port. This is useful when running automated integration tests to ensure that any concurrently running tests don’t clash on a hardcoded port number.

But there’s more to the underlying server than just a port. One of the most common things you’ll need to do with the underlying container is to set it up to handle HTTPS requests. To do that, the first thing you must do is create a keystore using the JDK’s `keytool` command-line utility, as shown next:

```
$ keytool -keystore mykeys.jks -genkey -alias tomcat -keyalg RSA
```

You’ll be asked several questions about your name and organization, most of which are irrelevant. But when asked for a password, remember what you choose. For the sake of this example, I chose `letmein` as the password.

Next, you’ll need to set a few properties to enable HTTPS in the embedded server. You could specify them all on the command line, but that would be terribly inconvenient. Instead, you’ll probably set them in the `application.properties` or `application.yml` file. In `application.yml`, the properties might look like this:

```
server:
  port: 8443
  ssl:
    key-store: file:///path/to/mykeys.jks
    key-store-password: letmein
    key-password: letmein
```

Here the `server.port` property is set to 8443, a common choice for development HTTPS servers. The `server.ssl.key-store` property should be set to the path where the keystore file is created. Here it’s shown with a `file://` URL to load it from the filesystem, but if you package it within the application JAR file, you’ll use a `classpath:` URL to reference it. And both the `server.ssl.key-store-password` and `server.ssl.key-password` properties are set to the password that was given when creating the keystore.

With these properties in place, your application should be listening for HTTPS requests on port 8443. Depending on which browser you’re using, you may encounter a warning about the server not being able to verify its identity. This is nothing to worry about when serving from localhost during development.

6.1.4 Configuring logging

Most applications provide some form of logging. And even if your application doesn't log anything directly, the libraries that your application uses will certainly log their activity.

By default, Spring Boot configures logging via Logback (<http://logback.qos.ch>) to write to the console at an INFO level. You've probably already seen plenty of INFO-level entries in the application logs as you've run the application and other examples. But as a reminder, here's a logging sample showing the default log format (wrapped to fit within the page margins):

```
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
➡ com.example.demo.Hello           Here's a log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
➡ com.example.demo.Hello           Here's another log entry.
2021-07-29 17:24:24.187 INFO 52240 --- [nio-8080-exec-1]
➡ com.example.demo.Hello           And here's one more.
```

For full control over the logging configuration, you can create a logback.xml file at the root of the classpath (in src/main/resources). Here's an example of a simple logback.xml file you might use:

```
<configuration>
  <appender name="STDOUT" class="ch.qos.logback.core.ConsoleAppender">
    <encoder>
      <pattern>
        %d{HH:mm:ss.SSS} [%thread] %-5level %logger{36} - %msg%n
      </pattern>
    </encoder>
  </appender>
  <logger name="root" level="INFO"/>
  <root level="INFO">
    <appender-ref ref="STDOUT" />
  </root>
</configuration>
```

With this new configuration, the same sample log entries from earlier might look like this (wrapped to fit in the page margins):

```
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello -
Here's a log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello -
Here's another log entry.
17:25:09.088 [http-nio-8080-exec-1] INFO com.example.demo.Hello -
And here's one more.
```

Aside from the pattern used for logging, this Logback configuration is more or less equivalent to the default you'll get if you have no logback.xml file. But by editing logback.xml, you can gain full control over your application's log files.

NOTE The specifics of what can go into `logback.xml` are outside the scope of this book. Refer to Logback's documentation for more information.

The most common changes you'll make to a logging configuration are to change the logging levels and perhaps to specify a file where the logs should be written. With Spring Boot configuration properties, you can make those changes without having to create a `logback.xml` file.

To set the logging levels, you create properties that are prefixed with `logging.level`, followed by the name of the logger for which you want to set the logging level. For instance, suppose you'd like to set the root logging level to `WARN`, but log Spring Security logs at a `DEBUG` level. The following entries in `application.yml` will take care of that for you:

```
logging:
  level:
    root: WARN
  org:
    springframework:
      security: DEBUG
```

Optionally, you can collapse the Spring Security package name to a single line for easier reading as follows:

```
logging:
  level:
    root: WARN
    org.springframework.security: DEBUG
```

Now suppose that you want to write the log entries to the file `TacoCloud.log` at `/var/logs/`. The `logging.file.path` and `logging.file.name` properties can help achieve that, as shown next:

```
logging:
  file:
    path: /var/logs/
    file: TacoCloud.log
  level:
    root: WARN
  org:
    springframework:
      security: DEBUG
```

Assuming that the application has write permissions to `/var/logs/`, the log entries will be written to `/var/logs/TacoCloud.log`. By default, the log files rotate once they reach 10 MB in size.

6.1.5 Using special property values

When setting properties, you aren't limited to declaring their values as hardcoded String and numeric values. Instead, you can derive their values from other configuration properties.

For example, suppose (for whatever reason) you want to set a property named `greeting.welcome` to echo the value of another property named `spring.application.name`. To achieve this, you could use the `${}` placeholder markers when setting `greeting.welcome` as follows:

```
greeting:
  welcome: ${spring.application.name}
```

You can even embed that placeholder amid other text, as shown here:

```
greeting:
  welcome: You are using ${spring.application.name}.
```

As you've seen, configuring Spring's own components with configuration properties makes it easy to inject values into those components' properties and to fine-tune auto-configuration. Configuration properties aren't exclusive to the beans that Spring creates. With a small amount of effort, you can take advantage of configuration properties in your own beans. Let's see how.

6.2 Creating your own configuration properties

As I mentioned earlier, configuration properties are nothing more than properties of beans that have been designated to accept configurations from Spring's environment abstraction. What I didn't mention is how those beans are designated to consume those configurations.

To support property injection of configuration properties, Spring Boot provides the `@ConfigurationProperties` annotation. When placed on any Spring bean, it specifies that the properties of that bean can be injected from properties in the Spring environment.

To demonstrate how `@ConfigurationProperties` works, suppose that you've added the following method to `OrderController` to list the authenticated user's past orders:

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user));

    return "orderList";
}
```

Along with that, you've also added the next necessary `findByUserOrderByPlacedAtDesc()` method to `OrderRepository`:

```
List<Order> findByUserOrderByPlacedAtDesc(User user);
```

Notice that this repository method is named with a clause of `OrderByPlacedAtDesc`. The `OrderBy` portion specifies a property by which the results will be ordered—in this case, the `placedAt` property. The `Desc` at the end causes the ordering to be in descending order. Therefore, the list of orders returned will be sorted from most recent to least recent.

As written, this controller method may be useful after the user has placed a handful of orders, but it could become a bit unwieldy for the most avid of taco connoisseurs. A few orders displayed in the browser are useful; a never-ending list of hundreds of orders is just noise. Let's say that you want to limit the number of orders displayed to the most recent 20 orders. You can change `ordersForUser()` as follows:

```
@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, 20);
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
}
```

along with the corresponding changes to `OrderRepository`, shown next:

```
List<TacoOrder> findByUserOrderByPlacedAtDesc(
    User user, Pageable pageable);
```

Here you've changed the signature of the `findByUserOrderByPlacedAtDesc()` method to accept a `Pageable` as a parameter. `Pageable` is Spring Data's way of selecting some subset of the results by a page number and page size. In the `ordersForUser()` controller method, you constructed a `PageRequest` object that implemented `Pageable` to request the first page (page zero) with a page size of 20 to get up to 20 of the most recently placed orders for the user.

Although this works fantastically, it leaves me a bit uneasy that you've hardcoded the page size. What if you later decide that 20 is too many orders to list, and you decide to change it to 10? Because it's hardcoded, you'd have to rebuild and redeploy the application.

Rather than hardcode the page size, you can set it with a custom configuration property. First, you need to add a new property called `pageSize` to `OrderController`, and then annotate `OrderController` with `@ConfigurationProperties` as shown in the next listing.

Listing 6.1 Enabling configuration properties in OrderController

```
@Controller
@RequestMapping("/orders")
@SessionAttributes("order")
@ConfigurationProperties(prefix="taco.orders")
public class OrderController {

    private int pageSize = 20;

    public void setPageSize(int pageSize) {
        this.pageSize = pageSize;
    }

    ...
    @GetMapping
    public String ordersForUser(
        @AuthenticationPrincipal User user, Model model) {

        Pageable pageable = PageRequest.of(0, pageSize);
        model.addAttribute("orders",
            orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));
        return "orderList";
    }
}
```

The most significant change made in listing 6.1 is the addition of the `@ConfigurationProperties` annotation. Its `prefix` attribute is set to `taco.orders`, which means that when setting the `pageSize` property, you need to use a configuration property named `taco.orders.pageSize`.

The new `pageSize` property defaults to 20, but you can easily change it to any value you want by setting a `taco.orders.pageSize` property. For example, you could set this property in `application.yml` like this:

```
taco:
  orders:
    pageSize: 10
```

Or, if you need to make a quick change while in production, you can do so without having to rebuild and redeploy the application by setting the `taco.orders.pageSize` property as an environment variable as follows:

```
$ export TACO_ORDERS_PAGESIZE=10
```

Any means by which a configuration property can be set can be used to adjust the page size of the recent orders page. Next, we'll look at how to set configuration data in property holders.

6.2.1 Defining configuration property holders

There's nothing that says `@ConfigurationProperties` must be set on a controller or any other specific kind of bean. `@ConfigurationProperties` are in fact often placed on beans whose sole purpose in the application is to be holders of configuration data. This keeps configuration-specific details out of the controllers and other application classes. It also makes it easy to share common configuration properties among several beans that may make use of that information.

In the case of the `pageSize` property in `OrderController`, you could extract it to a separate class. The following listing uses the `OrderProps` class in such a way.

Listing 6.2 Extracting `pageSize` to a holder class

```
package tacos.web;
import org.springframework.boot.context.properties.
    ConfigurationProperties;
import org.springframework.stereotype.Component;
import lombok.Data;

@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
public class OrderProps {

    private int pageSize = 20;
}
```

As you did with `OrderController`, the `pageSize` property defaults to 20, and `OrderProps` is annotated with `@ConfigurationProperties` to have a prefix of `taco.orders`. It's also annotated with `@Component` so that Spring component scanning will automatically discover it and create it as a bean in the Spring application context. This is important, because the next step is to inject the `OrderProps` bean into `OrderController`.

There's nothing particularly special about configuration property holders. They're beans that have their properties injected from the Spring environment. They can be injected into any other bean that needs those properties. For `OrderController`, this means removing the `pageSize` property from `OrderController` and instead injecting and using the `OrderProps` bean, as shown next:

```
private OrderProps props;

public OrderController(OrderRepository orderRepo,
    OrderProps props) {
    this.orderRepo = orderRepo;
    this.props = props;
}

...
```

```

@GetMapping
public String ordersForUser(
    @AuthenticationPrincipal User user, Model model) {

    Pageable pageable = PageRequest.of(0, props.getPageSize());
    model.addAttribute("orders",
        orderRepo.findByUserOrderByPlacedAtDesc(user, pageable));

    return "orderList";
}

```

Now `OrderController` is no longer responsible for handling its own configuration properties. This keeps the code in `OrderController` slightly neater and allows you to reuse the properties in `OrderProps` in any other bean that may need them. Moreover, you're collecting configuration properties that pertain to orders in one place: the `OrderProps` class. If you need to add, remove, rename, or otherwise change the properties therein, you need to apply those changes only in `OrderProps`. And for testing purposes, it's easy to set configuration properties directly on a test-specific `OrderProps` and give it to the controller prior to the test.

For example, let's pretend that you're using the `pageSize` property in several other beans when you decide it would be best to apply some validation to that property to limit its values to no less than 5 and no more than 25. Without a holder bean, you'd have to apply validation annotations to `OrderController`, the `pageSize` property, and all other classes using that property. But because you've extracted `pageSize` into `OrderProps`, you only must make the changes to `OrderProps`, as shown here:

```

package tacos.web;
import javax.validation.constraints.Max;
import javax.validation.constraints.Min;

import org.springframework.boot.context.properties.
    ConfigurationProperties;
import org.springframework.stereotype.Component;
import org.springframework.validation.annotation.Validated;

import lombok.Data;

@Component
@ConfigurationProperties(prefix="taco.orders")
@Data
@Validated
public class OrderProps {

    @Min(value=5, message="must be between 5 and 25")
    @Max(value=25, message="must be between 5 and 25")
    private int pageSize = 20;

}

```

Although you could as easily apply the `@Validated`, `@Min`, and `@Max` annotations to `OrderController` (and any other beans that can be injected with `OrderProps`), it

would just clutter up `OrderController` that much more. With a configuration property holder bean, you’ve collected configuration property specifics in one place, leaving the classes that need those properties relatively clean.

6.2.2 Declaring configuration property metadata

Depending on your IDE, you may have noticed that the `taco.orders.pageSize` entry in `application.yml` (or `application.properties`) has a warning saying something like Unknown Property ‘taco’. This warning appears because there’s missing metadata concerning the configuration property you just created. Figure 6.2 shows what this looks like when I hover over the `taco` portion of the property in the Spring Tool Suite.

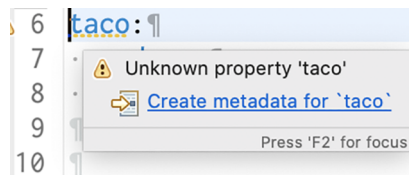


Figure 6.2 A warning for missing configuration property metadata

Configuration property metadata is completely optional and doesn’t prevent configuration properties from working. But the metadata can be useful for providing some minimal documentation around the configuration properties, especially in the IDE. For example, when I hover over the `spring.security.user.password` property, I see what’s shown in figure 6.3. Although the hover help you get is minimal, it can be enough to help understand what the property is used for and how to use it.

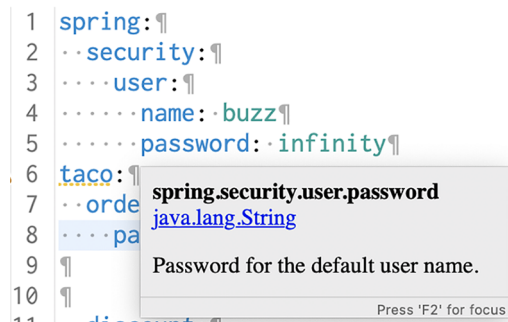


Figure 6.3 Hover documentation for configuration properties in the Spring Tool Suite

To help those who might use the configuration properties that you define—which might even be you—it’s generally a good idea to create some metadata around those properties. At least it gets rid of those annoying yellow warnings in the IDE.

To create metadata for your custom configuration properties, you’ll need to create a file under the `META-INF` (e.g., in the project under `src/main/resources/META-INF`) named `additional-spring-configuration-metadata.json`.